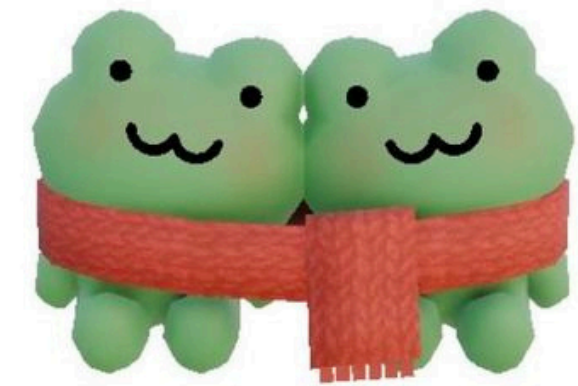


Streams, Lambdas & More Design Patterns

★ Software Design & Architecture ★



agenda



- SOLID recap
- Streams and lambdas
- Strategy Pattern
- Observer Pattern



- Assignment 1 is over! (if you submitted on time)
- Assignment 2 Part 1 spec is out, repos should be released by Saturday latest
 - ‘Two parts’ to assignment-ii but they’re not related at all
- Feedback form

some admin



recall



What does SOLID stand for?
Name the five principles and
explain each one.

1. **Single Responsibility Principle (SRP)**
2. **Open/Closed Principle (OCP)**
3. **Liskov's Substitution Principle (LSP)**
4. **Interface Segregation**
5. **Dependency Inversion**



summary of
SOLID

streams and lambdas

```
public static void main(String[] args) {  
    List<String> names = Arrays.asList("alice", "bob", "charlie");  
    List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);  
  
    List<String> upperNames = names.stream()  
        .map(String::toUpperCase)  
        .collect(Collectors.toList());  
  
    List<Integer> squares = numbers.stream()  
        .map(n -> n * n)  
        .collect(Collectors.toList());  
}
```

- Java's way of enabling **declarative, functional programming**.
 - Different from the imperative programming you're likely used to
- Like a function, this paradigm only concerns itself by transforming input into output.
- Common stream methods: `forEach`, `filter`, `map`, `reduce`

Lambda Expressions

- Concise way to represent anonymous functions
 - (parameters) -> expression
 - (parameters) -> { statements; }

Streams

- Sequence of elements supporting functional-style operations
- Process collections declaratively (similar to SQL queries)
- Two types of operations:
 - Intermediate: filter(), map(), sorted() (return a stream)
 - Terminal: forEach(), collect(), count() (produce result)

streams & lambdas



streams & lambdas

Why bother?

- Eliminates common bugs
 - Example: using `forEach` makes index out of bounds impossible
- Often lead to more compact and readable code
- Encourages immutability (don't modify source)
- Easy parallel processing with `parallelStream()`

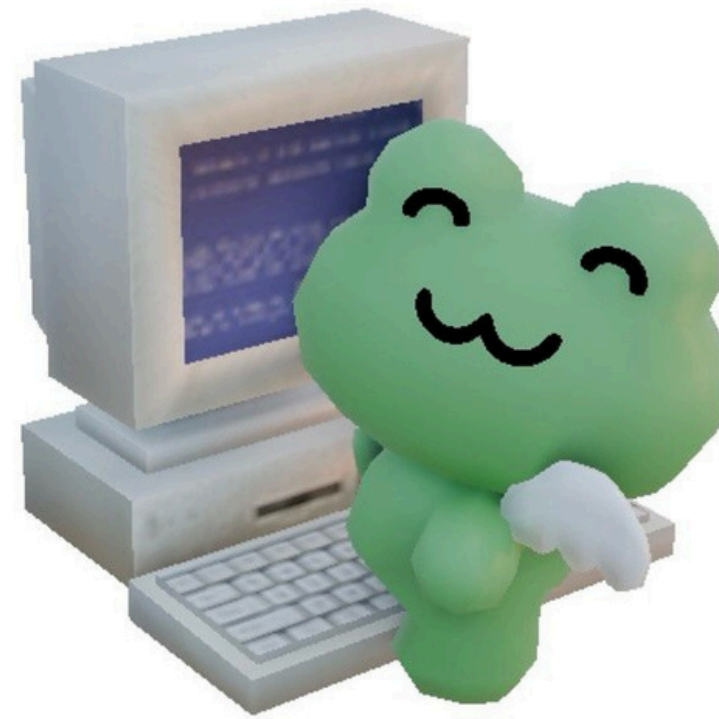


streams & lambdas



```
List<String> strings = new ArrayList<String>(Arrays.asList(new String[] { "1", "2", "3", "4", "5" }));  
  
strings.stream().forEach(x -> System.out.println(x));  
  
strings.stream().forEach(x -> {  
    System.out.println(x);  
});
```

- **x**: A parameter representing the element in the stream
- **→**: indicates and declares the anonymous function body. This is a short form for a single-line instruction.
- **{ }**: alternative way of writing a lambda. The braces declares a code scope so you can have multi-line instructions.



code demo!

Stream



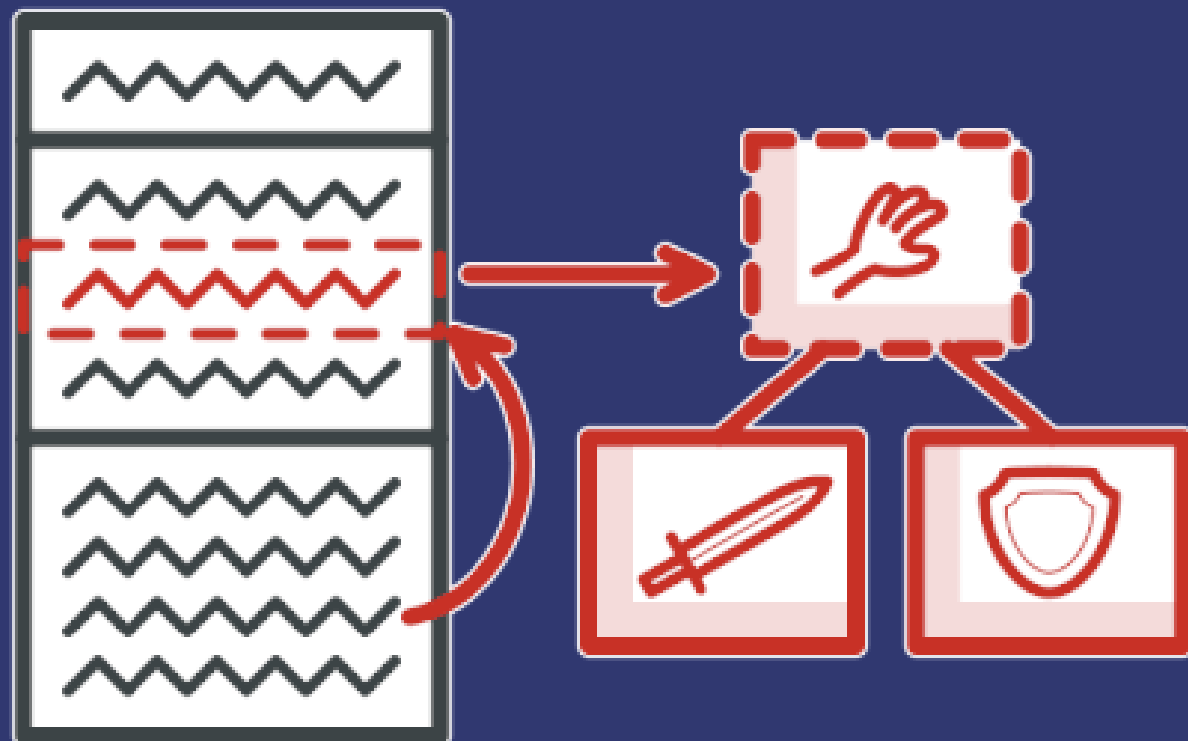


strategy pattern

Behavioural

What is it?

- Behavioural design pattern that defines a family of algorithms
- Encapsulates each algorithm in its own class
- Makes algorithms interchangeable at runtime



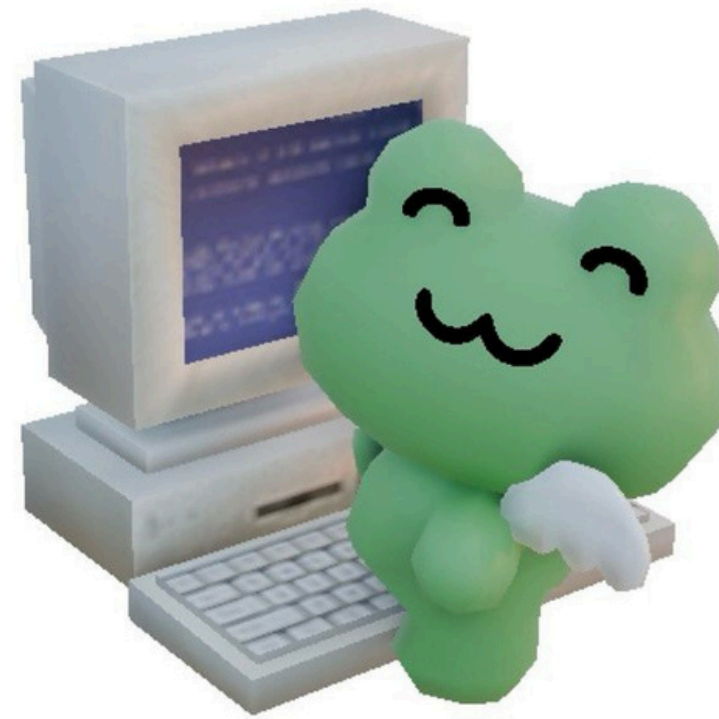
Why use it?

- Eliminates massive conditional statements (if/else or switch)
- Follows OCP
- Isolates algorithm logic from client code

When use it?

- Multiple ways to do the same thing (sorting, validation, pricing, etc.)
- You need to switch algorithms dynamically
- You want to avoid polluting a class with variant behavior

strategy pattern



code demo!

Restaurant





observer pattern

Behavioural



What is it?

- Behavioral pattern where objects (observers) watch another object (subject).
- Subject automatically notifies observers when its state changes

Why use it?

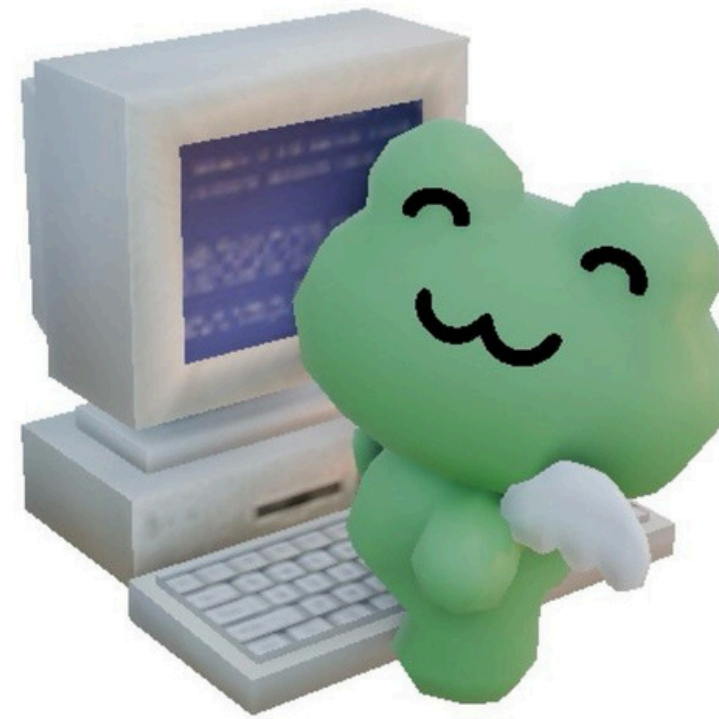
- Decouples subject from observers
- Supports one-to-many dependencies cleanly
- Observers can join/leave at runtime without changing subject

When use it?

- Changes to one object require changing others
- Multiple unknown objects should be notified about an event



observer pattern



code demo!

YouTube





lab time!

SitarME306